

Semantic Analysis

Attribute Grammars

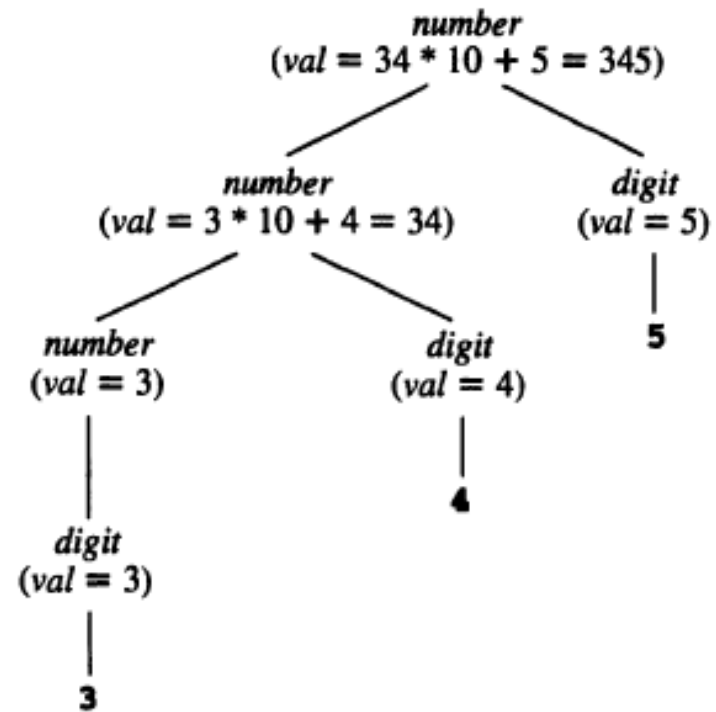
- Idea: associate *attributes* with each node in the (abstract) syntax tree
- Examples of attributes
 - Type information
 - Storage location
 - Assignable (e.g., expression vs variable; lvalue vs rvalue for C/C++ programmers)
 - Value (for constant expressions)
 - etc. ...
- Notation: $X.a$ if a is an attribute of node X

Table 6.1

Attribute grammar for
Example 6.1

Grammar Rule	Semantic Rules
$number_1 \rightarrow$ $number_2 \text{ digit}$	$number_1.val =$ $number_2.val * 10 + digit.val$
$number \rightarrow digit$	$number.val = digit.val$
$digit \rightarrow 0$	$digit.val = 0$
$digit \rightarrow 1$	$digit.val = 1$
$digit \rightarrow 2$	$digit.val = 2$
$digit \rightarrow 3$	$digit.val = 3$
$digit \rightarrow 4$	$digit.val = 4$
$digit \rightarrow 5$	$digit.val = 5$
$digit \rightarrow 6$	$digit.val = 6$
$digit \rightarrow 7$	$digit.val = 7$
$digit \rightarrow 8$	$digit.val = 8$
$digit \rightarrow 9$	$digit.val = 9$

Figure 6.1
Parse tree showing
attribute computations for
Example 6.1



Example 6.2

Consider the following grammar for simple integer arithmetic expressions:

$$\begin{aligned} \text{exp} &\rightarrow \text{exp} + \text{term} \mid \text{exp} - \text{term} \mid \text{term} \\ \text{term} &\rightarrow \text{term} * \text{factor} \mid \text{factor} \\ \text{factor} &\rightarrow (\text{exp}) \mid \mathbf{number} \end{aligned}$$

This grammar is a slightly modified version of the simple expression grammar studied extensively in previous chapters. The principal attribute of an *exp* (or *term* or *factor*) is its numeric value, which we write as *val*. The attribute equations for the *val* attribute are given in Table 6.2.

Table 6.2

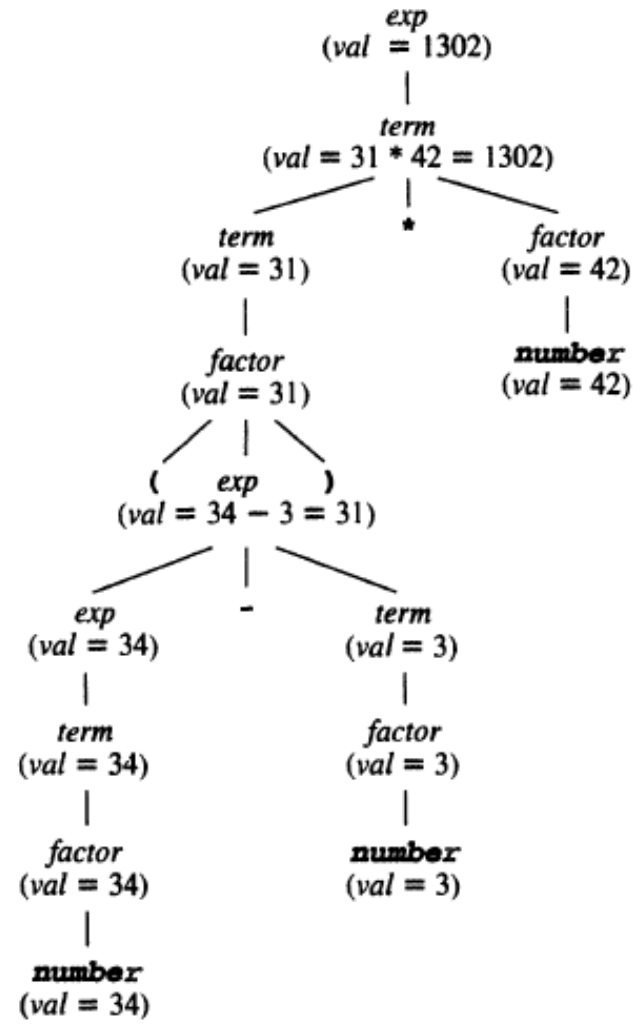
Attribute grammar for
Example 6.2

Grammar Rule	Semantic Rules
$\text{exp}_1 \rightarrow \text{exp}_2 + \text{term}$	$\text{exp}_1.\text{val} = \text{exp}_2.\text{val} + \text{term.val}$
$\text{exp}_1 \rightarrow \text{exp}_2 - \text{term}$	$\text{exp}_1.\text{val} = \text{exp}_2.\text{val} - \text{term.val}$
$\text{exp} \rightarrow \text{term}$	$\text{exp.val} = \text{term.val}$
$\text{term}_1 \rightarrow \text{term}_2 * \text{factor}$	$\text{term}_1.\text{val} = \text{term}_2.\text{val} * \text{factor.val}$
$\text{term} \rightarrow \text{factor}$	$\text{term.val} = \text{factor.val}$
$\text{factor} \rightarrow (\text{exp})$	$\text{factor.val} = \text{exp.val}$
$\text{factor} \rightarrow \mathbf{number}$	$\text{factor.val} = \mathbf{number.val}$

Figure 6.2

Parse tree for $(34-3)*42$

showing *val* attribute
computations for the
attribute grammar of
Example 6.2



Example 6.3

Consider the following simple grammar of variable declarations in a C-like syntax:

$$\begin{aligned} \text{decl} &\rightarrow \text{type } \text{var-list} \\ \text{type} &\rightarrow \text{int} \mid \text{float} \\ \text{var-list} &\rightarrow \text{id}, \text{var-list} \mid \text{id} \end{aligned}$$

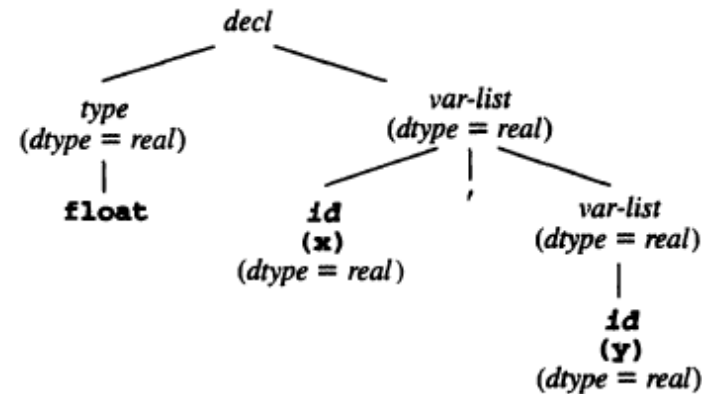
Table 6.3

Attribute grammar for
Example 6.3

Grammar Rule	Semantic Rules
$decl \rightarrow type\ var\text{-}list$	$var\text{-}list.dtype = type.dtype$
$type \rightarrow \mathbf{int}$	$type.dtype = integer$
$type \rightarrow \mathbf{float}$	$type.dtype = real$
$var\text{-}list_1 \rightarrow id, var\text{-}list_2$	$id.dtype = var\text{-}list_1.dtype$ $var\text{-}list_2.dtype = var\text{-}list_1.dtype$
$var\text{-}list \rightarrow id$	$id.dtype = var\text{-}list.dtype$

Figure 6.3

Parse tree for the string
float x,y showing the
dtype attribute as specified
by the attribute grammar of
Table 6.3.



Example 6.4

Consider a modification of the number grammar of Example 6.1, where numbers may be octal or decimal. Suppose this is indicated by a one-character suffix **o** (for octal) or **d** (for decimal). Then we have the following grammar:

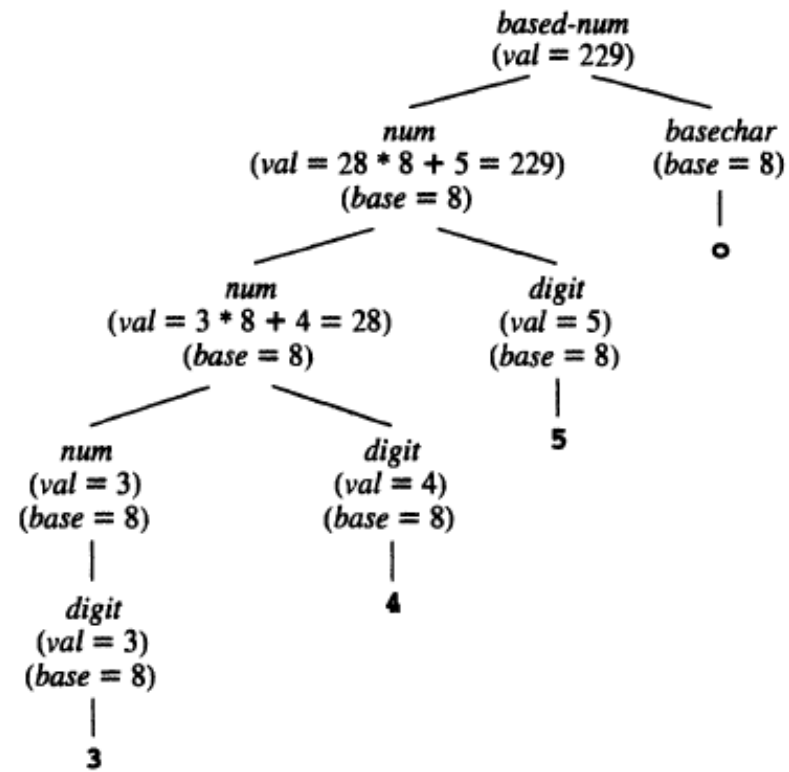
$$\textit{based-num} \rightarrow \textit{num basechar}$$
$$\textit{basechar} \rightarrow \mathbf{o} \mid \mathbf{d}$$
$$\textit{num} \rightarrow \textit{num digit} \mid \textit{digit}$$
$$\textit{digit} \rightarrow \mathbf{0} \mid \mathbf{1} \mid \mathbf{2} \mid \mathbf{3} \mid \mathbf{4} \mid \mathbf{5} \mid \mathbf{6} \mid \mathbf{7} \mid \mathbf{8} \mid \mathbf{9}$$

Table 6.4

Attribute grammar for
Example 6.4

Grammar Rule	Semantic Rules
$based_num \rightarrow num\ basechar$	$based_num.val = num.val$ $num.base = basechar.base$
$basechar \rightarrow o$	$basechar.base = 8$
$basechar \rightarrow d$	$basechar.base = 10$
$num_1 \rightarrow num_2\ digit$	$num_1.val =$ if $digit.val = error$ or $num_2.val = error$ then $error$ else $num_2.val * num_1.base + digit.val$ $num_2.base = num_1.base$ $digit.base = num_1.base$
$num \rightarrow digit$	$num.val = digit.val$ $digit.base = num.base$
$digit \rightarrow 0$	$digit.val = 0$
$digit \rightarrow 1$	$digit.val = 1$
...	...
$digit \rightarrow 7$	$digit.val = 7$
$digit \rightarrow 8$	$digit.val =$ if $digit.base = 8$ then $error$ else 8
$digit \rightarrow 9$	$digit.val =$ if $digit.base = 8$ then $error$ else 9

Figure 6.4
Parse tree showing
attribute computations for
Example 6.4



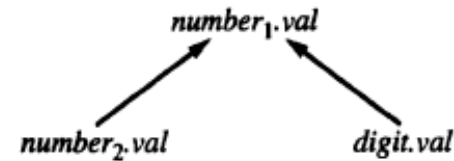
Dependency Graph

Example 6.6

Consider the grammar of Example 6.1, with the attribute grammar as given in Table 6.1. There is only one attribute, *val*, so for each symbol there is only one node in each dependency graph, corresponding to its *val* attribute. The grammar rule choice $number_1 \rightarrow number_2 digit$ has the single associated attribute equation

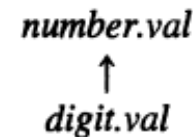
$$number_1.val = number_2.val * 10 + digit.val$$

The dependency graph for this grammar rule choice is



(In future dependency graphs, we will omit the subscripts for repeated symbols, since the graphical representation clearly distinguishes the different occurrences as different nodes.)

Similarly, the dependency graph for the grammar rule $number \rightarrow digit$ with the attribute equation $number.val = digit.val$ is



Example 6.2

Consider the following grammar for simple integer arithmetic expressions:

$$\begin{aligned} \text{exp} &\rightarrow \text{exp} + \text{term} \mid \text{exp} - \text{term} \mid \text{term} \\ \text{term} &\rightarrow \text{term} * \text{factor} \mid \text{factor} \\ \text{factor} &\rightarrow (\text{exp}) \mid \mathbf{number} \end{aligned}$$

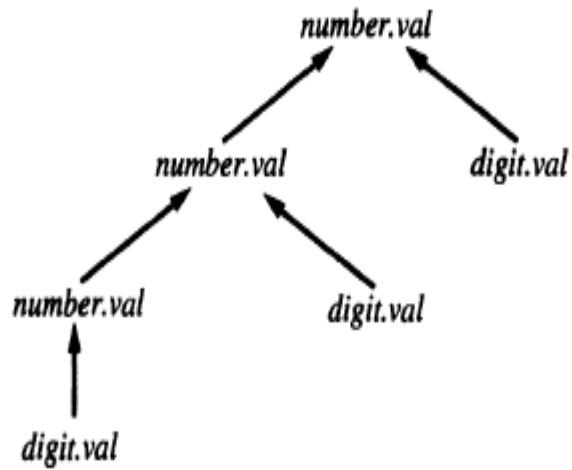
This grammar is a slightly modified version of the simple expression grammar studied extensively in previous chapters. The principal attribute of an *exp* (or *term* or *factor*) is its numeric value, which we write as *val*. The attribute equations for the *val* attribute are given in Table 6.2.

Table 6.2

Attribute grammar for
Example 6.2

Grammar Rule	Semantic Rules
$\text{exp}_1 \rightarrow \text{exp}_2 + \text{term}$	$\text{exp}_1.\text{val} = \text{exp}_2.\text{val} + \text{term.val}$
$\text{exp}_1 \rightarrow \text{exp}_2 - \text{term}$	$\text{exp}_1.\text{val} = \text{exp}_2.\text{val} - \text{term.val}$
$\text{exp} \rightarrow \text{term}$	$\text{exp.val} = \text{term.val}$
$\text{term}_1 \rightarrow \text{term}_2 * \text{factor}$	$\text{term}_1.\text{val} = \text{term}_2.\text{val} * \text{factor.val}$
$\text{term} \rightarrow \text{factor}$	$\text{term.val} = \text{factor.val}$
$\text{factor} \rightarrow (\text{exp})$	$\text{factor.val} = \text{exp.val}$
$\text{factor} \rightarrow \mathbf{number}$	$\text{factor.val} = \mathbf{number.val}$

Finally, the string **345** has the following dependency graph corresponding to its parse tree (see Figure 6.1):



§

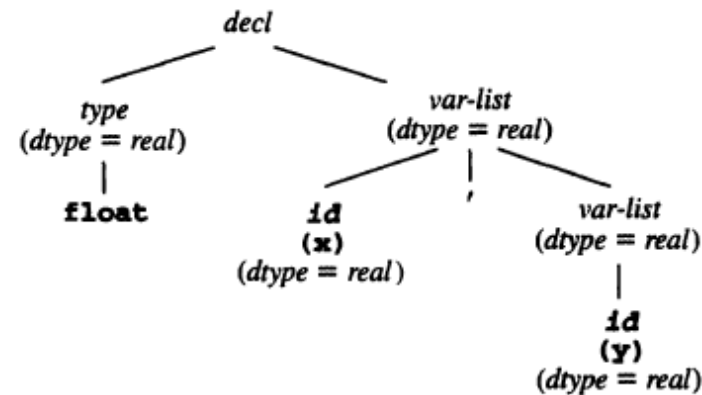
Table 6.3

Attribute grammar for
Example 6.3

Grammar Rule	Semantic Rules
$decl \rightarrow type\ var\text{-}list$	$var\text{-}list.dtype = type.dtype$
$type \rightarrow \mathbf{int}$	$type.dtype = integer$
$type \rightarrow \mathbf{float}$	$type.dtype = real$
$var\text{-}list_1 \rightarrow id, var\text{-}list_2$	$id.dtype = var\text{-}list_1.dtype$ $var\text{-}list_2.dtype = var\text{-}list_1.dtype$
$var\text{-}list \rightarrow id$	$id.dtype = var\text{-}list.dtype$

Figure 6.3

Parse tree for the string
float x,y showing the
dtype attribute as specified
by the attribute grammar of
Table 6.3.

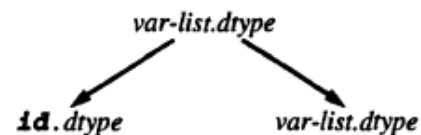


Example 6.7

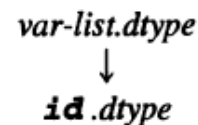
Consider the grammar of Example 6.3, with the attribute grammar for the attribute *dtype* given in Table 6.3. In this example the grammar rule $\text{var-list}_1 \rightarrow \mathbf{id}$, var-list_2 has the two associated attribute equations

$$\begin{aligned}\mathbf{id}.dtype &= \text{var-list}_1.dtype \\ \text{var-list}_2.dtype &= \text{var-list}_1.dtype\end{aligned}$$

and the dependency graph



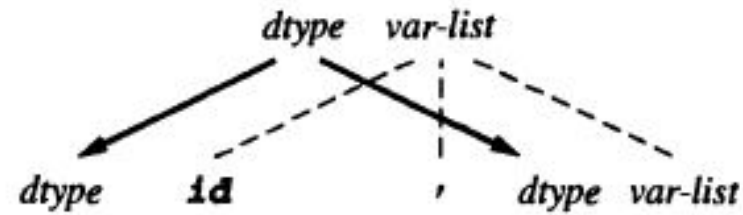
Similarly, the grammar rule $\text{var-list} \rightarrow \mathbf{id}$ has the dependency graph



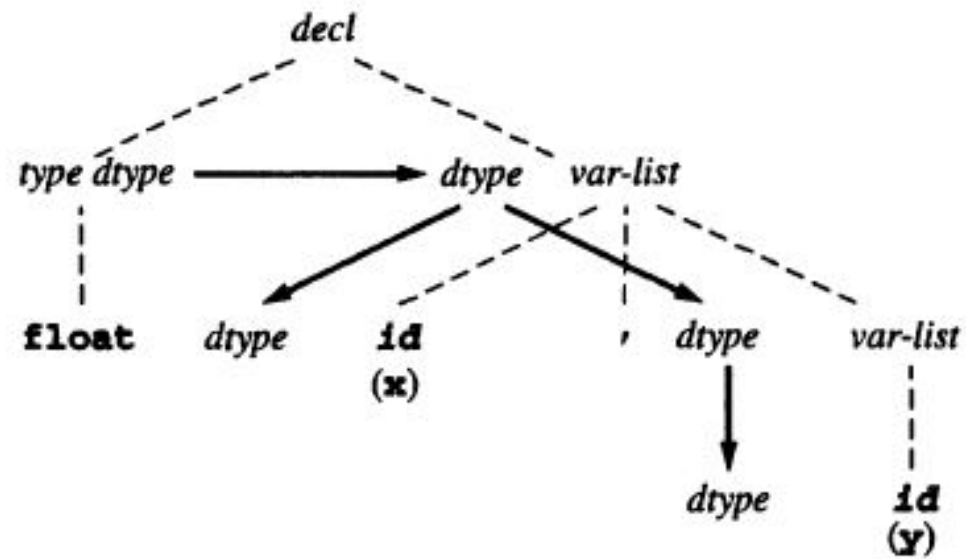
The two rules $\text{type} \rightarrow \mathbf{int}$ and $\text{type} \rightarrow \mathbf{float}$ have trivial dependency graphs.

Finally, the rule $\text{decl} \rightarrow \text{type var-list}$ with the associated equation $\text{var-list.dtype} = \text{type.dtype}$ has the dependency graph





Finally, the dependency graph for the string **float x,y** is



Example 6.8

Consider the grammar of based numbers of Example 6.4, with the attribute grammar for the attributes *base* and *val* as given in Table 6.4. We will draw the dependency graphs for the four grammar rules

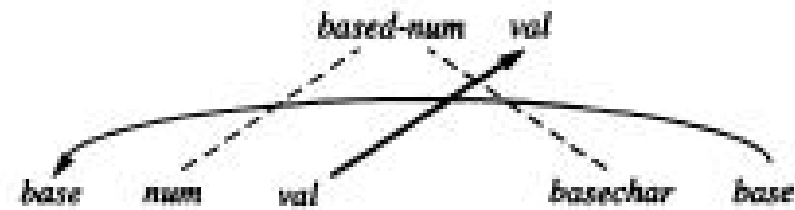
based-num $\rightarrow$ *num* *basechar*
num $\rightarrow$ *num* *digit*
num $\rightarrow$ *digit*
digit $\rightarrow$ **9**

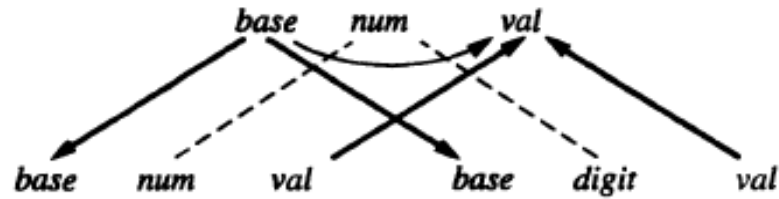
Table 6.4

Attribute grammar for
Example 6.4

Grammar Rule	Semantic Rules
$based_num \rightarrow num\ basechar$	$based_num.val = num.val$ $num.base = basechar.base$
$basechar \rightarrow 0$	$basechar.base = 8$
$basechar \rightarrow d$	$basechar.base = 10$
$num_1 \rightarrow num_2\ digit$	$num_1.val =$ if $digit.val = error$ or $num_2.val = error$ then $error$ else $num_2.val * num_1.base + digit.val$ $num_2.base = num_1.base$ $digit.base = num_1.base$
$num \rightarrow digit$	$num.val = digit.val$ $digit.base = num.base$
$digit \rightarrow 0$	$digit.val = 0$
$digit \rightarrow 1$	$digit.val = 1$
...	...
$digit \rightarrow 7$	$digit.val = 7$
$digit \rightarrow 8$	$digit.val =$ if $digit.base = 8$ then $error$ else 8
$digit \rightarrow 9$	$digit.val =$ if $digit.base = 8$ then $error$ else 9

We begin with the dependency graph for the grammar rule $\text{based-num} \rightarrow \text{num basechar}$.



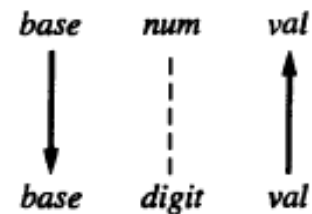


This graph expresses the dependencies of the three attribute equations

```

num1.val =
    if digit.val = error or num2.val = error
    then error
    else num2.val * num1.base + digit.val
num2.base = num1.base
digit.base = num1.base
  
```

The dependency graph for the grammar rule $num \rightarrow digit$ is similar:



Finally, we draw the dependency graph for the grammar rule $digit \rightarrow 9$:

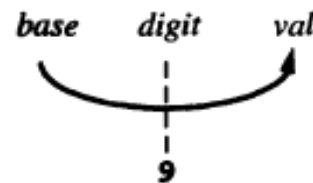


Figure 6.6

Dependency graph for the string 3450 (Example 6.8)

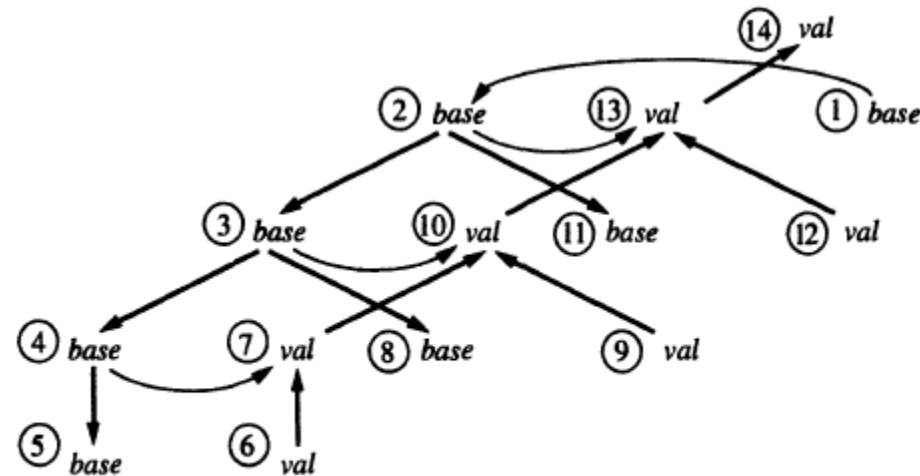
Example 6.9

The dependency graph of Figure 6.6 is a DAG. In Figure 6.7 we number the nodes of the graph (and erase the underlying parse tree for ease of visualization). One topological sort is given by the node order in which the nodes are numbered. Another topological sort is given by the order

12 6 9 1 2 11 3 8 4 5 7 10 13 14

Figure 6.7

Dependency graph for the string 3450 (Example 6.9)



§

Synthesized Attribute

An attribute is **synthesized** if all its dependencies point from child to parent in the parse tree. Equivalently, an attribute a is synthesized if, given a grammar rule $A \rightarrow X_1X_2 \dots X_n$, the only associated attribute equation with an a on the left-hand side is of the form

$$A.a = f(X_1.a_1, \dots, X_1.a_k, \dots, X_n.a_1, \dots, X_n.a_k)$$

An attribute grammar in which all the attributes are synthesized is called an **S-attributed grammar**.

Given that a parse or syntax tree has been constructed by a parser, the attribute values of an S-attributed grammar can be computed by a single bottom-up, or postorder, traversal of the tree. This can be expressed by the following pseudocode for a recursive postorder attribute evaluator:

```

typedef enum {Plus,Minus,Times} OpKind;
typedef enum {OpKind,ConstKind} ExpKind;
typedef struct streenode
{
    ExpKind kind;
    OpKind op;
    struct streenode *lchild,*rchild;
    int val;
} STreeNode;
typedef STreeNode *SyntaxTree;

```

the PostEval pseudocode would translate into the C code of Figure 6.8 for a left-to-right traversal.

Figure 6.8

C code for the postorder
attribute evaluator for
Example 6.11

```

void postEval(SyntaxTree t)
{
    int temp;
    if (t->kind == OpKind)
    {
        postEval(t->lchild);
        postEval(t->rchild);
        switch (t->op)
        {
            case Plus:
                t->val = t->lchild->val + t->rchild->val;
                break;
            case Minus:
                t->val = t->lchild->val - t->rchild->val;
                break;
            case Times:
                t->val = t->lchild->val * t->rchild->val;
                break;
        } /* end switch */
    } /* end if */
} /* end postEval */

```

Inherited Attribute

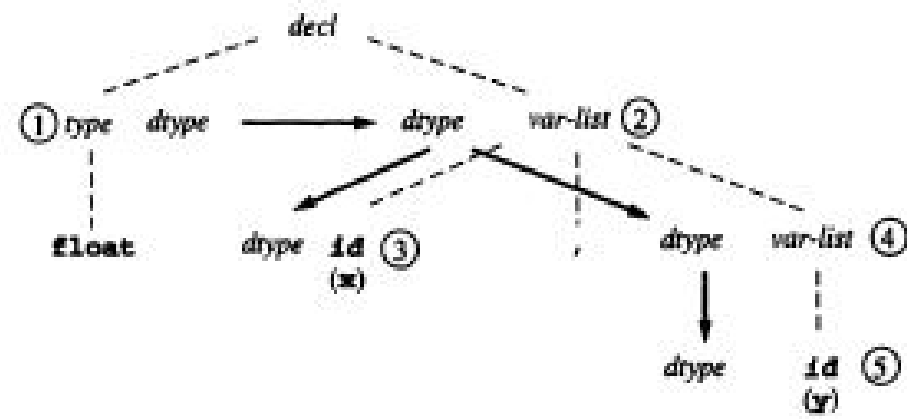
Definition

An attribute that is not synthesized is called an **inherited** attribute.

Figure 6.9
Different kinds of inherited
dependencies



Figure 6.10
Parse tree showing traversal
order for Example 6.12



```

typedef enum {decl,type,id} nodekind;
typedef enum {integer,real} typekind;
typedef struct treeNode
{
    nodekind kind;
    struct treeNode
        * lchild, * rchild, * sibling;
    typekind dtype;
    /* for type and id nodes */
    char * name;
    /* for id nodes only */
} * SyntaxTree;

```

The *EvalType* procedure now has the corresponding C code:

```

void evalType (SyntaxTree t)
{
    switch (t->kind)
    {
        case decl:
            t->rchild->dtype = t->lchild->dtype;
            evalType(t->rchild);
            break;
        case id:
            if (t->sibling != NULL)
            {
                t->sibling->dtype = t->dtype;
                evalType(t->sibling);
            }
            break;
    }
    /* end switch */
}
/* end evalType */

```

Example 6.13

Consider the grammar of Example 6.4 (page 266), which has the inherited attribute *base* (the dependency graphs are given in Example 6.8, page 273). We repeat the grammar of that example here:

$$\begin{aligned} \text{based-num} &\rightarrow \text{num basechar} \\ \text{basechar} &\rightarrow \text{○} \mid \text{d} \\ \text{num} &\rightarrow \text{num digit} \mid \text{digit} \\ \text{digit} &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

This grammar has two new features. First, there are two attributes, the synthesized attribute *val*, and the inherited attribute *base*, on which *val* depends. Second, the *base* attribute is inherited from the right child to the left child of a *based-num* (that is, from *basechar* to *num*). Thus, in this case, we must evaluate the children of a *based-num* from right to left instead of left to right. We proceed to give the pseudocode for an *EvalWithBase* procedure that computes both *base* and *val*. In this case, *base* is computed in preorder and *val* in postorder during a single pass (we discuss the question of multiple attributes and passes shortly). The pseudocode is as follows (see the attribute grammar in Table 6.4, page 267).

```

procedure EvalWithBase ( T: treenode );
begin
  case nodekind of T of
    based-num:
      EvalWithBase ( right child of T );
      assign base of right child of T to base of left child;
      EvalWithBase ( left child of T );
      assign val of left child of T to T.val;
    num:
      assign T.base to base of left child of T;
      EvalWithBase ( left child of T );
      if right child of T is not nil then
        assign T.base to base of right child of T;
        EvalWithBase ( right child of T );
        if vals of left and right children  $\neq$  error then
          T.val := T.base*(val of left child) + val of right child
        else T.val := error;
      else T.val := val of left child;

```

```

basechar:
  if child of T = o then T.base := 8
  else T.base := 10;
digit:
  if T.base = 8 and child of T = 8 or 9 then T.val := error
  else T.val := numval ( child of T );
end case;
end EvalWithBase;

```

We leave to the exercises the construction of C declarations for an appropriate syntax tree and a translation of the pseudocode for *EvalWithBase* into C code. §

In attribute grammars with combinations of synthesized and inherited attributes, if the synthesized attributes depend on the inherited attributes (as well as other synthesized attributes) but the inherited attributes do not depend on any synthesized attributes, then it is possible to compute all the attributes in a single pass over the parse or syntax tree. The previous example is a good example of how this is done, and the order of evaluation may be summarized by combining the *PostEval* and *PreEval* pseudocode procedures:

```

procedure CombinedEval ( T: treenode );
begin
  for each child C of T do
    compute all inherited attributes of C;
    CombinedEval ( C );
    compute all synthesized attributes of T;
  end;
end;

```

Situations in which inherited attributes depend on synthesized attributes are more complex and require more than one pass over the parse or syntax tree, as the following example shows.

Attributes as parameters and return values

6.2.3 Attributes as Parameters and Returned Values

Often when computing attributes, it makes sense to use parameters and returned function values to communicate attribute values rather than to store them as fields in a syntax tree record structure. This is particularly true if many of the attribute values are the same or are only used temporarily to compute other attribute values. In this case, it

In more complex situations, for instance when more than one synthesized attribute must be returned, it may be necessary to use a record structure or union as a returned value, or one may split up the recursive procedure into several procedures to cover different cases. We illustrate this with an example.

```

function EvalWithBase ( T: treenode ): integer;
var temp, temp2: integer;
begin
  case nodekind of T of
    based-num:
      SetBase ( right child of T );
      return EvalWithBase ( left child of T );
    num:
      temp := EvalWithBase ( left child of T );
      if right child of T is not nil then
        temp2 := EvalWithBase ( right child of T );
        if temp ≠ error and temp2 ≠ error then
          return base*temp + temp2
        else return error;
      else return temp;
    digit:
      if base = 8 and child of T = 8 or 9 then return error
      else return numval ( child of T );
  end case;
end EvalWithBase;

procedure SetBase ( T: treenode );
begin
  if child of T = ○ then base := 8
  else base := 10;
end SetBase ;

```

Grammar Rule	Semantic Rules
$based_num \rightarrow num\ basechar$	$based_num.val = num.val$
$basechar \rightarrow o$	$base := 8$
$basechar \rightarrow d$	$base := 10$
$num_1 \rightarrow num_2\ digit$	$num_1.val =$ if $digit.val = error$ or $num_2.val = error$ then $error$ else $num_2.val * base + digit.val$
etc.	etc.

Now *base* is no longer an attribute in the sense in which we have used it up to now, and the semantic rules no longer form an attribute grammar. Nevertheless, if *base* is known to be a variable with the appropriate properties, then these rules still adequately define the semantics of a *based-num* for the compiler writer. §

Example 6.17

Consider the attribute grammar of simple declarations of Table 6.3 (page 266), and the attribute evaluation procedure for this attribute grammar given in Example 6.12 (page 279). Typically, the information in declarations is inserted into a symbol table using the declared identifiers as keys and stored there for later lookup during the translation of other parts of the program. Let us, therefore, assume for this attribute grammar that a symbol table exists that will store the identifier name together with its declared data type and that name-data type pairs are inserted into the symbol table via an *insert* procedure declared as follows:

procedure *insert* (*name*: string; *dtype*: typekind);

Thus, instead of storing the data type of each variable in the syntax tree, we insert it into the symbol table using this procedure. Also, since each declaration has only one associated type, we can use a nonlocal variable to store the constant *dtype* of each declaration during processing. The resulting semantic rules are as follows:

Grammar Rule	Semantic Rules
<i>decl</i> → <i>type var-list</i>	
<i>type</i> → int	<i>dtype</i> = integer
<i>type</i> → float	<i>dtype</i> = real
<i>var-list</i> ₁ → id , <i>var-list</i> ₂	<i>insert</i> (id .name, <i>dtype</i>)
<i>var-list</i> → id	<i>insert</i> (id .name, <i>dtype</i>)

Computation of Attribute During Parsing

For synthesized attributes this is not a restriction, since the children of a node can be processed in arbitrary order, in particular from left to right. But, for inherited attributes, this means that there may be no “backward” dependencies in the dependency graph (dependencies pointing from right to left in the parse tree). For example, the attribute grammar of Example 6.4 (page 266) violates this property, since a *based-num* has its

L-Attribute

Definition

An attribute grammar for attributes $a_1, \dots, a_k$ is **L-attributed** if, for each inherited attribute a_j and each grammar rule

$$X_0 \rightarrow X_1 X_2 \dots X_n$$

the associated equations for a_j are all of the form

$$X_i.a_j = f_{ij}(X_0.a_1, \dots, X_0.a_k, X_1.a_1, \dots, X_1.a_k, \dots, X_{i-1}.a_1, \dots, X_{i-1}.a_k)$$

That is, the value of a_j at X_i can only depend on attributes of the symbols $X_0, \dots, X_{i-1}$ that occur to the left of X_i in the grammar rule.

Table 6.8

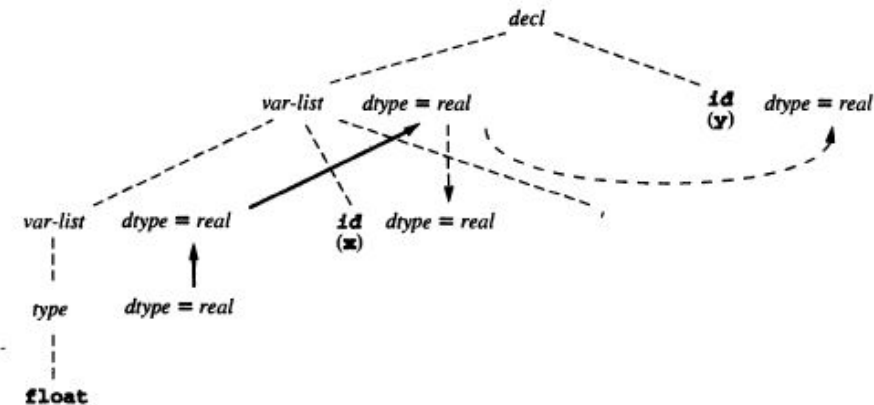
Parsing and semantic actions
for the expression $3*4+5$
during an LR parse

	Parsing Stack	Input	Parsing Action	Value Stack	Semantic Action
1	\$	3*4+5 \$	shift	\$	
2	\$ n	*4+5 \$	reduce $E \rightarrow n$	\$ n	$E.val = n.val$
3	\$ E	*4+5 \$	shift	\$ 3	
4	\$ E *	4+5 \$	shift	\$ 3 *	
5	\$ E * n	+5 \$	reduce $E \rightarrow n$	\$ 3 * n	$E.val = n.val$
6	\$ E * E	+5 \$	reduce $E \rightarrow E * E$	\$ 3 * 4	$E_1.val =$ $E_2.val * E_3.val$
7	\$ E	+5 \$	shift	\$ 12	
8	\$ E +	5 \$	shift	\$ 12 +	
9	\$ E + n	\$	reduce $E \rightarrow n$	\$ 12 + n	$E.val = n.val$
10	\$ E + E	\$	reduce $E \rightarrow E + E$	\$ 12 + 5	$E_1.val =$ $E_2.val + E_3.val$
11	\$ E	\$		\$ 17	

Dependence of Attribute Computation on the Syntax

Grammar Rule	Semantic Rules
$decl \rightarrow var\text{-}list\ id$	$id.dtype = var\text{-}list.dtype$
$var\text{-}list_1 \rightarrow var\text{-}list_2\ id\ ,$	$var\text{-}list_1.dtype = var\text{-}list_2.dtype$ $id.dtype = var\text{-}list_1.dtype$
$var\text{-}list \rightarrow type$	$var\text{-}list.dtype = type.dtype$
$type \rightarrow int$	$type.dtype = integer$
$type \rightarrow float$	$type.dtype = real$

We illustrate how this change in the grammar affects the parse tree and the *dtype* attribute computation in Figure 6.11, which shows the parse tree for the string **float x, y**, together with the attribute values and dependencies. The dependencies of the two *id.dtype* values on the values of the parent or sibling are drawn as dashed lines in the figure. While these dependencies appear to be violations of the claim that there are no inherited attributes in this attribute grammar, in fact these dependencies are always to leaves in the parse tree (that is, not recursive) and may be achieved by operations at the appropriate parent nodes. Thus, these dependencies are not viewed as inheritances. §



Type Expressions and Type Constructors

Type expressions denote the type of language constructs (variables, functions, etc.).

Basic Types (Primitive):

- Atomic types provided by the language:
 - integer, real, boolean, char, void
- Examples:
 - integer: Represents whole numbers (e.g., 42).

Type constructors

Type Constructors: Build complex types from basic types.

Constructor	Syntax	Example
Array	<code>array(index, type)</code>	<code>array(1..10, integer)</code> → Array of 10 integers
Record	<code>record(field: type)</code>	<code>record(address: integer, lexeme: array[1..15] of char)</code> → Struct with integer and char array fields
Pointer	<code>pointer(type)</code>	<code>pointer(integer)</code> → Pointer to an integer
Function	<code>domain → range</code>	<code>(integer, integer) → real</code> → Function taking two integers, returning a real
Union	<code>union(type1, type2)</code>	<code>union((i: integer), (f: float))</code> → Value can be integer or float

Type Names and Type Declarations

User-defined aliases for existing types, created using typedef.

- Why Use Them?
 - Increase code readability
 - Simplify complex type usage
 - Make code easier to maintain

- Examples:

```
typedef unsigned long ulong;
```

```
typedef int Coordinate;
```

Usage: `ulong fileSize; Coordinate x, y;`

- Benefit: Now, `ulong` and `Coordinate` can be used just like built-in types, making declarations clearer and less error-prone.

Struct Type Names and Declarations

- Struct Type Name:

Used to define complex data structures with a single name.

- Example:

```
typedef struct { char name[20]; int id; float gpa; }
```

```
Student;
```

Usage:

```
Student s1, s2; s1.id = 1001;
```

Recursive Type

A type defined in terms of itself, commonly used for linked lists and trees.

- C Example (Linked List Node):

```
typedef struct Node
{ int data;
  struct Node *next; }
Node;
```

Type Equivalence

- Type Equivalence:
Determines when two types are considered the same by the compiler.
- Two principal forms:
 - Name Equivalence: Types are equivalent if they have the same name or were declared in the same declaration.
 - Structural Equivalence: Types are equivalent if their structures are built using the same type constructors from the same basic types.

Figure 6.19

A simple grammar for
type expressions

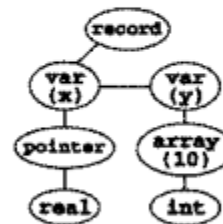
```
var-decls → var-decls ; var-decl | var-decl  
var-decl → id : type-exp  
type-exp → simple-type | structured-type  
simple-type → int | bool | real | char | void  
structured-type → array [num] of type-exp |  
                  record var-decls end |  
                  union var-decls end |  
                  pointer to type-exp |  
                  proc( type-exps ) type-exp  
type-exps → type-exps , type-exp | type-exp
```

in this figure.

Consider, first, the type expression

```
record
  x: pointer to real;
  y: array [10] of int
end
```

This type expression can be represented by the syntax tree



Here the child nodes of the record are represented as a sibling list, since the number of components of a record is arbitrary. Note that the nodes representing simple types form the leaves of the tree.

Figure 6.20

Pseudocode for a *typeEqual* function that tests structural equivalence of type expressions from the grammar of Figure 6.19

```

function typeEqual ( t1, t2 : TypeExp ) : Boolean;
var temp : Boolean ;
    p1, p2 : TypeExp ;
begin
    if t1 and t2 are of simple type then return t1 = t2
    else if t1.kind = array and t2.kind = array then
        return t1.size = t2.size and typeEqual ( t1.child1, t2.child1 )
    else if t1.kind = record and t2.kind = record
        or t1.kind = union and t2.kind = union then
        begin
            p1 := t1.child1 ;
            p2 := t2.child1 ;
            temp := true ;
            while temp and p1 ≠ nil and p2 ≠ nil do
                if p1.name ≠ p2.name then
                    temp := false
                else if not typeEqual ( p1.child1 , p2.child1 )
                then temp := false
                else begin
                    p1 := p1.sibling ;
                    p2 := p2.sibling ;
                end;
            return temp and p1 = nil and p2 = nil ;
        end
    else if t1.kind = pointer and t2.kind = pointer then
        return typeEqual ( t1.child1 , t2.child1 )
    else if t1.kind = proc and t2.kind = proc then
        begin
            p1 := t1.child1 ;
            p2 := t2.child1 ;
            temp := true ;
            while temp and p1 ≠ nil and p2 ≠ nil do
                if not typeEqual ( p1.child1 , p2.child1 )
                then temp := false
                else begin
                    p1 := p1.sibling ;
                    p2 := p2.sibling ;
                end;
            return temp and p1 = nil and p2 = nil
                and typeEqual ( t1.child2 , t2.child2 )
        end
    else return false ;
end ; (* typeEqual *)

```

Recursive Types and Type Equivalence

- Recursive Type Equivalence:
Recursive types can pose challenges for structural equivalence, as the type checker must recognize equivalence even when types refer to themselves.
- `struct NodeA { int data; struct NodeA* next; };`

Type inference and Type Checking

Figure 6.22

A simple grammar to
illustrate type checking

```
program → var-decls ; stmts  
var-decls → var-decls ; var-decl | var-decl  
var-decl → id : type-exp  
type-exp → int | bool | array [num] of type-exp  
stmts → stmts ; stmt | stmt  
stmt → if exp then stmt | id := exp
```

Table 6.10

Attribute grammar for type
checking of the simple
grammar of Figure 6.22

Grammar Rule	Semantic Rules
$var\text{-}decl \rightarrow id : type\text{-}exp$	$insert(id.name, type\text{-}exp.type)$
$type\text{-}exp \rightarrow int$	$type\text{-}exp.type := integer$
$type\text{-}exp \rightarrow bool$	$type\text{-}exp.type := boolean$
$type\text{-}exp_1 \rightarrow array$ $[num] \text{ of } type\text{-}exp_2$	$type\text{-}exp_1.type :=$ $makeTypeNode(array, num.size,$ $type\text{-}exp_2.type)$
$stmt \rightarrow if \ exp \ then \ stmt$	if not $typeEqual(exp.type, boolean)$ then $type\text{-}error(stmt)$
$stmt \rightarrow id := exp$	if not $typeEqual(lookup(id.name),$ $exp.type)$ then $type\text{-}error(stmt)$
$exp_1 \rightarrow exp_2 + exp_3$	if not ($typeEqual(exp_2.type, integer)$ and $typeEqual(exp_3.type, integer)$) then $type\text{-}error(exp_1)$; $exp_1.type := integer$
$exp_1 \rightarrow exp_2 \text{ or } exp_3$	if not ($typeEqual(exp_2.type, boolean)$ and $typeEqual(exp_3.type, boolean)$) then $type\text{-}error(exp_1)$; $exp_1.type := boolean$
$exp_1 \rightarrow exp_2 [exp_3]$	if $isArrayType(exp_2.type)$ and $typeEqual(exp_3.type, integer)$ then $exp_1.type := exp_2.type.child1$ else $type\text{-}error(exp_1)$
$exp \rightarrow num$	$exp.type := integer$
$exp \rightarrow true$	$exp.type := boolean$
$exp \rightarrow false$	$exp.type := boolean$
$exp \rightarrow id$	$exp.type := lookup(id.name)$